МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчёт по лабораторной работе №1
“Основные конструкции языка Python ”

Выполнил:
Студент группы ИУ5-35Б
Сергучанов Т.Н.
Преподаватель:
Гапанюк Ю.Е.
Нардид А.Н.

Москва 2026

Задание:

Разработать программы для решения биквадратного уравнения.

1.Программы должна быть разработаны в виде консольного приложения на языке Python. Разработайте три программы на языке Python:

i.с применением процедурной парадигмы (использование функций, оператор def)

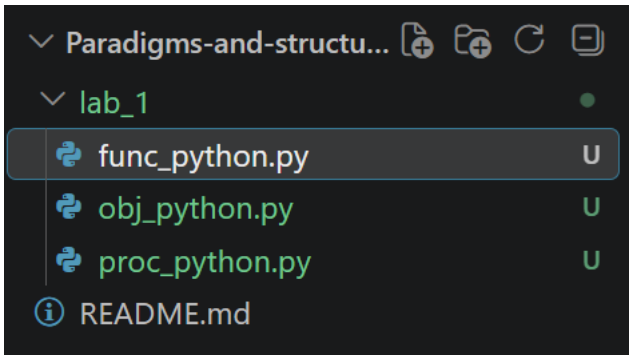
ii.с применением объектно-ориентированной парадигмы (использование классов)

iii.с применением функциональной парадигмы (сопоставление с образцом)

2.Каждая программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и ДЕЙСТВИТЕЛЬНЫЕ корни уравнения (в зависимости от дискриминанта).

3.Коэффициенты A, B, C могут быть заданы в виде параметров командной строки. Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2.

4.Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.



Lab_1 – папка лабораторной работы №1

func_python.py – файл с программой, написанной с применением функциональной парадигмы

obj_python.py – файл с программой, написанной с применением объектно-ориентированной парадигмы

proc_python.py – файл с программой, написанной с применением процедурной парадигмы

i.Текст программы с применением процедурной парадигмы

```
import math

def get_coef(prompt):
  print(prompt)
  while True:
    try:
      coef_str = input()
      coef = float(coef_str)
      return coef

    except(IndexError, ValueError):
      print(prompt)

def get_roots(a, b, c):

  result = []

  D = b**2 - 4*a*c
  if D == 0:
    root_y = -b/(2.0*a)
    root1 = math.sqrt(root_y)
    root2 = -math.sqrt(root_y)
    result.append(root1)
    result.append(root2)
  if D > 0:
    root_y1 = (-b - math.sqrt(D))/(2.0*a)
    root_y2 = (-b + math.sqrt(D))/(2.0*a)
    if root_y1 >=0:
      root1 = math.sqrt(root_y1)
      root2 = -root1
      result.append(root1)
      result.append(root2)
    if root_y2 >= 0:
      root3 = math.sqrt(root_y2)
      root4 = -root3
      result.append(root3)
      result.append(root4)
  unique_result = list(set(result))
  return unique_result

def print_roots():
  a = get_coef("Введите коэффициент A")
  b = get_coef("Введите коэффициент B")
  c = get_coef("Введите коэффициент C")

  roots = get_roots(a, b, c)

  if not roots:
```

```
    print("Корней нет")
  else:
    i = 0
    while i != len(roots):
      print(f"x{1 + i} = {roots[i]}\n")
      i += 1



def main():
  print_roots()

if __name__ == "__main__":
  main()
```

ii.Текст программы с применением объектно-ориентированной парадигмы

```
import math

class BiquadraticSolver:

  def __init__(self):
    self.a = 0
    self.b = 0
    self.c = 0

  def input_data(self, prompt):
    print(prompt)
    while True:
      try:
        coef_str = input()
        coef = float(coef_str)
        return coef
      except(IndexError, ValueError):
        print(prompt)

  def solve (self):
    result = []
    D = self.b**2 - 4*self.a*self.c
    if D == 0:
      root_y = -self.b/(2.0*self.a)
      root1 = math.sqrt(root_y)
      root2 = -math.sqrt(root_y)
      result.append(root1)
      result.append(root2)
    if D > 0:
      root_y1 = (-self.b - math.sqrt(D))/(2.0*self.a)
      root_y2 = (-self.b + math.sqrt(D))/(2.0*self.a)
```

```
      if root_y1 >=0:
        root1 = math.sqrt(root_y1)
        root2 = -root1
        result.append(root1)
        result.append(root2)
      if root_y2 >= 0:
        root3 = math.sqrt(root_y2)
        root4 = -root3
        result.append(root3)
        result.append(root4)
    unique_result = list(set(result))
    return unique_result

  def get_roots(self):
    self.a = self.input_data("Введите коэффициент A")
    self.b = self.input_data("Введите коэффициент B")
    self.c = self.input_data("Введите коэффициент C")

    roots = self.solve()

    if not roots:
      print("Корней нет")
    else:
      i = 0
      while i != len(roots):
        print(f"x{1 + i} = {roots[i]}\n")
        i += 1

def main():
  solver = BiquadraticSolver()
  solver.get_roots()

if __name__ == "__main__":
  main()
```

iii.Текст программы с применением функциональной парадигмы

```
import math

def calculate_biquadratic_roots(a, b, c):
  result = []
  D = b**2 - 4 * a * c

  if D == 0:
    root_y = -b / (2.0 * a)
    root1 = math.sqrt(root_y)
    root2 = -math.sqrt(root_y)
    result.append(root1)
    result.append(root2)
```

```
  elif D > 0:
    root_y1 = (-b - math.sqrt(D)) / (2.0 * a)
    root_y2 = (-b + math.sqrt(D)) / (2.0 * a)

    if root_y1 >= 0:
      result.append(math.sqrt(root_y1))
      result.append(-math.sqrt(root_y1))
    if root_y2 >= 0:
      result.append(math.sqrt(root_y2))
      result.append(-math.sqrt(root_y2))

  return list(set(result))

def safe_input_float(prompt):
  print(prompt)
  while True:
    try:
      return float(input())
    except ValueError:
      print(prompt)

def main(input_func, solver_func):

  a = input_func("Введите коэффициент A: ")
  b = input_func("Введите коэффициент B: ")
  c = input_func("Введите коэффициент C: ")

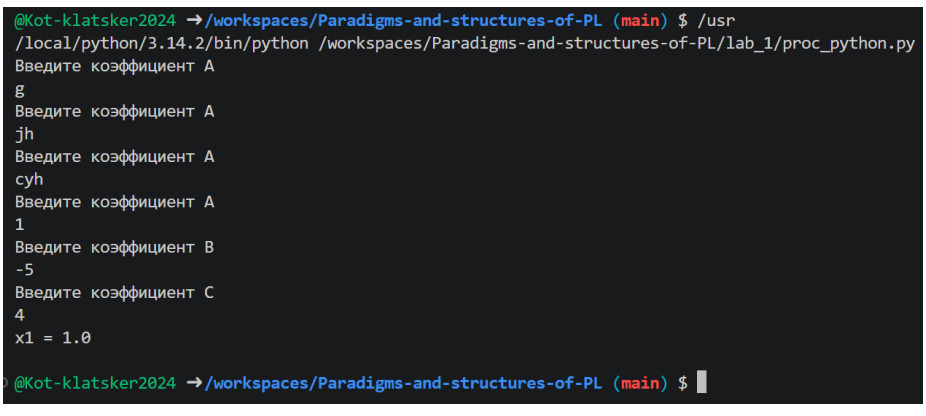
  roots = solver_func(a, b, c)

  if not roots:
    print("\nКорней нет")
  else:
    for i, root in enumerate(roots):
      print(f"x{i + 1} = {root}\n")

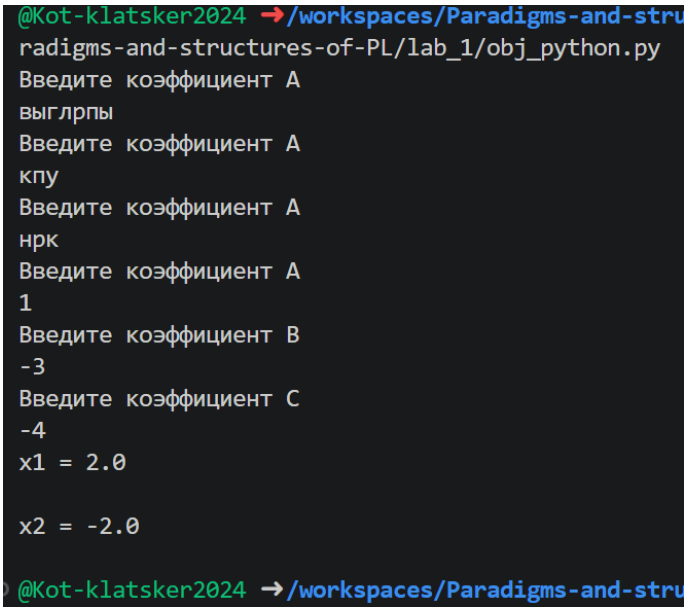
if __name__ == "__main__":
  main(safe_input_float, calculate_biquadratic_roots)
```

Скриншоты выполнения программы

Процедурная парадигма



Объектно-ориентированная парадигма



Функциональная парадигма

